

Mega MoE: Fused Expert-Parallel Execution

Deep Dive into DeepSeek — Chapter 27

Yin Yang

Foundation Books Project

The one rule of this chapter

Fusion changes *the schedule and the interval that must be measured*.

It does **not** change which weighted expert contribution each source token is owed.

- A correct expert-parallel path can still serialize dispatch, two expert linear layers, activation, and combine behind separate launch boundaries.
- **Mega MoE** is DeepGEMM's fused, multi-rank, persistent SM100 path over symmetric peer memory that attacks exactly that systems problem.

The one rule of this chapter

Fusion changes *the schedule and the interval that must be measured*.
It does **not** change which weighted expert contribution each source token is owed.

- A correct expert-parallel path can still serialize dispatch, two expert linear layers, activation, and combine behind separate launch boundaries.
- **Mega MoE** is DeepGEMM's fused, multi-rank, persistent SM100 path over symmetric peer memory that attacks exactly that systems problem.

Goal: follow one six-token, three-rank trace through ownership, scheduling, completion, precision, and evidence — and see where each piece lives in the pinned DeepGEMM source.

1. Why fuse the expert-parallel path

Roadmap

1. Why fuse the expert-parallel path
2. Output equivalence and route weights

Roadmap

1. Why fuse the expert-parallel path
2. Output equivalence and route weights
3. Symmetric memory and route ownership

Roadmap

1. Why fuse the expert-parallel path
2. Output equivalence and route weights
3. Symmetric memory and route ownership
4. The fused interval and role overlap

Roadmap

1. Why fuse the expert-parallel path
2. Output equivalence and route weights
3. Symmetric memory and route ownership
4. The fused interval and role overlap
5. Scheduling: ring capacity and task publication

Roadmap

1. Why fuse the expert-parallel path
2. Output equivalence and route weights
3. Symmetric memory and route ownership
4. The fused interval and role overlap
5. Scheduling: ring capacity and task publication
6. Completion: grid and rank barriers

Roadmap

1. Why fuse the expert-parallel path
2. Output equivalence and route weights
3. Symmetric memory and route ownership
4. The fused interval and role overlap
5. Scheduling: ring capacity and task publication
6. Completion: grid and rank barriers
7. SM100 recipes and kernel plans

Roadmap

1. Why fuse the expert-parallel path
2. Output equivalence and route weights
3. Symmetric memory and route ownership
4. The fused interval and role overlap
5. Scheduling: ring capacity and task publication
6. Completion: grid and rank barriers
7. SM100 recipes and kernel plans
8. Correctness, benchmarking, and evidence

Roadmap

1. Why fuse the expert-parallel path
2. Output equivalence and route weights
3. Symmetric memory and route ownership
4. The fused interval and role overlap
5. Scheduling: ring capacity and task publication
6. Completion: grid and rank barriers
7. SM100 recipes and kernel plans
8. Correctness, benchmarking, and evidence
9. Integration, portability, and failure modes

Roadmap

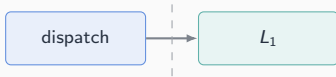
1. Why fuse the expert-parallel path
2. Output equivalence and route weights
3. Symmetric memory and route ownership
4. The fused interval and role overlap
5. Scheduling: ring capacity and task publication
6. Completion: grid and rank barriers
7. SM100 recipes and kernel plans
8. Correctness, benchmarking, and evidence
9. Integration, portability, and failure modes
10. Summary and what comes next

Why fuse the expert-parallel path

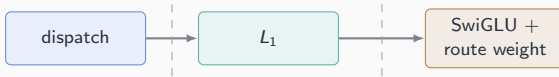
A convenient chain, and the debt it still owes

dispatch

A convenient chain, and the debt it still owes



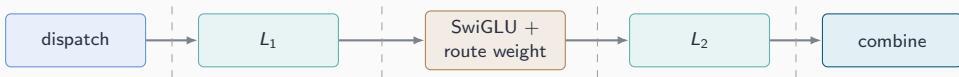
A convenient chain, and the debt it still owes



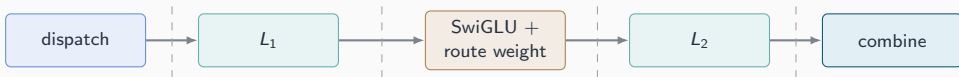
A convenient chain, and the debt it still owes



A convenient chain, and the debt it still owes

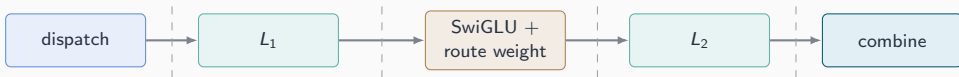


A convenient chain, and the debt it still owes



- Each dashed boundary above is a separate, correct launch boundary — and also a place a ready block can be held back for no functional reason.
- [Mega MoE](#) combines expert-parallel data movement, two expert linears, activation, route weights, and combine under a symmetric-memory, persistent-scheduling contract.

A convenient chain, and the debt it still owes



- Each dashed boundary above is a separate, correct launch boundary — and also a place a ready block can be held back for no functional reason.
- [Mega MoE](#) combines expert-parallel data movement, two expert linears, activation, route weights, and combine under a symmetric-memory, persistent-scheduling contract.

“Persistent” is a scheduling word, not a timing word: resident roles advance different routed blocks whenever their dependencies are satisfied, and no sooner.

Output equivalence and route weights

The oracle every rewrite must reproduce

$$y_t = \sum_r w_{t,r} L_{2,e_{t,r}}(\text{SwiGLU}(L_{1,e_{t,r}}(x_t)))$$

- Because L_2 is bias-free, linearity permits $w L_2(g) = L_2(w g)$ — Mega MoE applies the weight in the L_1 epilogue.

The oracle every rewrite must reproduce

$$y_t = \sum_r w_{t,r} L_{2,e_{t,r}}(\text{SwiGLU}(L_{1,e_{t,r}}(x_t)))$$

- Because L_2 is bias-free, linearity permits $w L_2(g) = L_2(w g)$ — Mega MoE applies the weight in the L_1 epilogue.
- An affine L_2 would need a correction term; the source path relies on bias-free L_2 instead.

The oracle every rewrite must reproduce

$$y_t = \sum_r w_{t,r} L_{2,e_{t,r}}(\text{SwiGLU}(L_{1,e_{t,r}}(x_t)))$$

- Because L_2 is bias-free, linearity permits $w L_2(g) = L_2(w g)$ — Mega MoE applies the weight in the L_1 epilogue.
- An affine L_2 would need a correction term; the source path relies on bias-free L_2 instead.

Every retained route owes exactly one tagged contribution — never omitted, never counted twice.

Symmetric memory and route ownership

Six tokens, three ranks: a fixture you can trace by hand

A *symmetric buffer*: same slice geometry on every rank, addressed by (rank, offset) — not equal payload values.

- Six retained routes, top-2 selection, three active experts out of six; receive counts (5, 4, 3) for (e_0, e_1, e_2) .

Six tokens, three ranks: a fixture you can trace by hand

A *symmetric buffer*: same slice geometry on every rank, addressed by (rank, offset) — not equal payload values.

- Six retained routes, top-2 selection, three active experts out of six; receive counts (5, 4, 3) for (e_0, e_1, e_2) .
- Source metadata (rank, token, route slot) travels with every byte, so a peer address stays computable *and* meaningful.

Six tokens, three ranks: a fixture you can trace by hand

A *symmetric buffer*: same slice geometry on every rank, addressed by (rank, offset) — not equal payload values.

- Six retained routes, top-2 selection, three active experts out of six; receive counts (5, 4, 3) for (e_0, e_1, e_2) .
- Source metadata (rank, token, route slot) travels with every byte, so a peer address stays computable *and* meaningful.

The router's output stays fixed all chapter. Only the mechanism tracing it changes.

The fused interval and role overlap

Strict inside a block, loose across blocks

- Inside one routed block: input visible before L_1 ; L_1 and its epilogue done before L_2 consumes the pool block; the remote contribution visible before rank completion; combine sees exactly the valid slots.

Strict inside a block, loose across blocks

- Inside one routed block: input visible before L_1 ; L_1 and its epilogue done before L_2 consumes the pool block; the remote contribution visible before rank completion; combine sees exactly the valid slots.
- Across blocks: pulling, computing, and writing back different blocks may all coexist.

Strict inside a block, loose across blocks

- Inside one routed block: input visible before L_1 ; L_1 and its epilogue done before L_2 consumes the pool block; the remote contribution visible before rank completion; combine sees exactly the valid slots.
- Across blocks: pulling, computing, and writing back different blocks may all coexist.

Source structure shows overlap is *possible*. Only a trace or timing record shows how much overlap a given run achieved.

Scheduling: ring capacity and task publication

Pool blocks and tasks, not expert waves

- **Ring-token capacity**: aligned pool slots for reusable activation storage — bounds physical storage, not context length or total logical routes.

Pool blocks and tasks, not expert waves

- **Ring-token capacity**: aligned pool slots for reusable activation storage — bounds physical storage, not context length or total logical routes.
- **Scheduler task**: a published record naming phase, expert, M -block, N -cluster, pool block, valid rows, and shape.

Pool blocks and tasks, not expert waves

- **Ring-token capacity**: aligned pool slots for reusable activation storage — bounds physical storage, not context length or total logical routes.
- **Scheduler task**: a published record naming phase, expert, M -block, N -cluster, pool block, valid rows, and shape.
- A superseded “expert-wave” width from earlier revisions is absent from the pinned mechanism this chapter teaches.

Pool blocks and tasks, not expert waves

- **Ring-token capacity**: aligned pool slots for reusable activation storage — bounds physical storage, not context length or total logical routes.
- **Scheduler task**: a published record naming phase, expert, M -block, N -cluster, pool block, valid rows, and shape.
- A superseded “expert-wave” width from earlier revisions is absent from the pinned mechanism this chapter teaches.

Full before consume; empty before reuse — one rule, protecting both task slots and activation pools.

Completion: grid and rank barriers

Local gate, then cross-rank signal

- A local grid gate answers whether *one block* may advance.

Local gate, then cross-rank signal

- A local grid gate answers whether *one block* may advance.
- A signed, two-location rank-barrier cycle answers whether *every rank* has made this call's contributions visible.

Local gate, then cross-rank signal

- A local grid gate answers whether *one block* may advance.
- A signed, two-location rank-barrier cycle answers whether *every rank* has made this call's contributions visible.

A missing arrival deadlocks; an early one releases combine before bytes are visible; a stale one can make the *next* call appear complete — three distinct failures, one generation-identity invariant.

SM100 recipes and kernel plans

Two recipes, one fused interval

	BF16-by-BF16	FP8-by-FP4
Routed input/weights	BF16	FP8 in, packed FP4 weights

Two recipes, one fused interval

	BF16-by-BF16	FP8-by-FP4
Routed input/weights	BF16	FP8 in, packed FP4 weights
$L_1 \rightarrow L_2$ state	BF16	FP8 + packed UE8M0 scales

Two recipes, one fused interval

	BF16-by-BF16	FP8-by-FP4
Routed input/weights	BF16	FP8 in, packed FP4 weights
$L_1 \rightarrow L_2$ state	BF16	FP8 + packed UE8M0 scales

Both recipes fuse the identical systems interval and share every scheduling and completion obligation. A *kernel plan* binds one recipe to concrete tile shapes, layouts, and thread-role splits — plan fields, not mathematical properties of MoE.

Correctness, benchmarking, and evidence

Inspected mechanism, source-reported benchmarks

- The repository test compares a non-overlapped modular composition against the fused endpoint: exact equality, or below 10^{-8} with shared experts.

Inspected mechanism, source-reported benchmarks

- The repository test compares a non-overlapped modular composition against the fused endpoint: exact equality, or below 10^{-8} with shared experts.
- Upstream benchmark rows remain *source-reported*, not locally reproduced.

Inspected mechanism, source-reported benchmarks

- The repository test compares a non-overlapped modular composition against the fused endpoint: exact equality, or below 10^{-8} with shared experts.
- Upstream benchmark rows remain *source-reported*, not locally reproduced.

The retained local devices were SM90, SM120, and SM110 — no SM100 device. No local Mega MoE correctness, latency, or speedup claim follows. “Not applicable” is an eligibility result, not a zero.

Integration, portability, and failure modes

What's portable, what's specific, what fails first

- DeepGEMM supplies the matrix machinery; DeepEP supplies a modular external oracle, not an internal call; EPLB may move where a logical expert physically lives.

What's portable, what's specific, what fails first

- DeepGEMM supplies the matrix machinery; DeepEP supplies a modular external oracle, not an internal call; EPLB may move where a logical expert physically lives.
- Portable: the output formula and one-contribution invariant. Architecture-specific: the SM100 matrix path, Tensor Memory, FP4 packing, thread split.

What's portable, what's specific, what fails first

- DeepGEMM supplies the matrix machinery; DeepEP supplies a modular external oracle, not an internal call; EPLB may move where a logical expert physically lives.
- Portable: the output formula and one-contribution invariant. Architecture-specific: the SM100 matrix path, Tensor Memory, FP4 packing, thread split.

Layer every integration test: identity, then layout, then fused output, then overlap, then — only then — timing.

Summary and what comes next

Concept-to-code map for the chapter

- **API and SM100 eligibility** → `DeepGEMM/csrc/apis/mega.hpp`

Concept-to-code map for the chapter

- **API and SM100 eligibility** → `DeepGEMM/csrc/apis/mega.hpp`
- **symmetric workspace layout** → `DeepGEMM/deep_gemm/include/deep_gemm/layout/mega_moe.cuh`

Concept-to-code map for the chapter

- **API and SM100 eligibility** → `DeepGEMM/csrc/apis/mega.hpp`
- **symmetric workspace layout** → `DeepGEMM/deep_gemm/include/deep_gemm/layout/mega_moe.cuh`
- **task-publication scheduler** → `DeepGEMM/deep_gemm/include/deep_gemm/scheduler/mega_moe.cuh`

Concept-to-code map for the chapter

- **API and SM100 eligibility** → `DeepGEMM/csrc/apis/mega.hpp`
- **symmetric workspace layout** → `DeepGEMM/deep_gemm/include/deep_gemm/layout/mega_moe.cuh`
- **task-publication scheduler** → `DeepGEMM/deep_gemm/include/deep_gemm/scheduler/mega_moe.cuh`
- **grid/rank barrier** → `DeepGEMM/deep_gemm/include/deep_gemm/comm/barrier.cuh`

Concept-to-code map for the chapter

- **API and SM100 eligibility** → `DeepGEMM/csrc/apis/mega.hpp`
- **symmetric workspace layout** → `DeepGEMM/deep_gemm/include/deep_gemm/layout/mega_moe.cuh`
- **task-publication scheduler** → `DeepGEMM/deep_gemm/include/deep_gemm/scheduler/mega_moe.cuh`
- **grid/rank barrier** → `DeepGEMM/deep_gemm/include/deep_gemm/comm/barrier.cuh`
- **repository correctness oracle** → `DeepGEMM/tests/test_mega_moe.py`

Concept-to-code map for the chapter

- **API and SM100 eligibility** → `DeepGEMM/csrc/apis/mega.hpp`
- **symmetric workspace layout** → `DeepGEMM/deep_gemm/include/deep_gemm/layout/mega_moe.cuh`
- **task-publication scheduler** → `DeepGEMM/deep_gemm/include/deep_gemm/scheduler/mega_moe.cuh`
- **grid/rank barrier** → `DeepGEMM/deep_gemm/include/deep_gemm/comm/barrier.cuh`
- **repository correctness oracle** → `DeepGEMM/tests/test_mega_moe.py`

Same debt, moved boundary: fusion relocates the schedule, never the function.

Next: EPLB changes where a logical expert lives; Mega MoE decides how one eligible rank group executes the resulting routed work.